

3 编写一维杆有限元程序

1、任务名称

编写一维杆有限元程序：从手算过程到 MATLAB 程序实现。

2、任务背景

有限元软件的本质是将有限元理论过程转化为程序算法。为了理解有限元计算的基本流程，避免只把软件当作黑箱使用，本任务基于一维杆单元理论，使用 MATLAB 编写一个简单的一维杆有限元求解程序，实现节点位移、单元应变、应力和内力的自动计算。

本程序主要完成从输入节点、单元、材料、载荷和边界条件，到组装总体刚度矩阵、施加边界条件、求解位移、计算单元结果的全过程。

3、任务目标

通过本任务，完成以下目标：

1. 理解一维杆有限元计算的基本流程；
2. 掌握节点坐标、单元连接、材料参数、载荷和边界条件的数据输入方法；
3. 编写单元刚度矩阵计算程序；
4. 实现总体刚度矩阵的自动组装；
5. 施加位移边界条件并求解节点位移；
6. 计算单元应变、应力和内力；
7. 通过两个算例验证程序正确性。

4、程序功能要求

本程序应实现以下功能：

1. 输入节点坐标；
2. 输入单元连接关系；
3. 输入材料参数 E 和截面面积 A ；
4. 输入节点载荷向量；
5. 输入位移约束条件；
6. 计算各单元刚度矩阵；
7. 组装总体刚度矩阵；
8. 施加边界条件；
9. 求解节点位移；
10. 计算单元应变、应力和内力；
11. 输出总体刚度矩阵、节点位移、支座反力和单元结果表。

5、程序设计思路

一维杆单元每个节点只有一个轴向位移自由度。对于任意一个二节点杆单元，其单元刚度矩阵为：

$$k_e = \frac{EA}{L_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \quad (3-1)$$

(3-1) 式中，E 为弹性模量，A 为截面面积，L_e 为单元长度。

程序首先读取节点坐标和单元连接关系，然后循环每个单元，计算单元长度和单元刚度矩阵。之后根据单元节点编号，将单元刚度矩阵叠加到总体刚度矩阵中。完成总体刚度矩阵组装后，根据给定约束划分自由自由度和约束自由度，求解未知节点位移。最后根据单元两端节点位移计算应变、应力和内力。

求解线性方程时使用 MATLAB 的反斜杠运算 A\B，该运算可用于求解 A*x = B 形式的线性方程组，见图 3-1。



图 3-1 思路流程图

一维杆有限元程序的基本计算过程为：输入节点、单元、材料、载荷和约束条件，计算单元刚度矩阵，并组装形成总体刚度矩阵。接着施加边界条件，求解节点位移，再根据位移计算支座反力、单元应变、应力和内力，最后输出计算结果，见图 3-2。

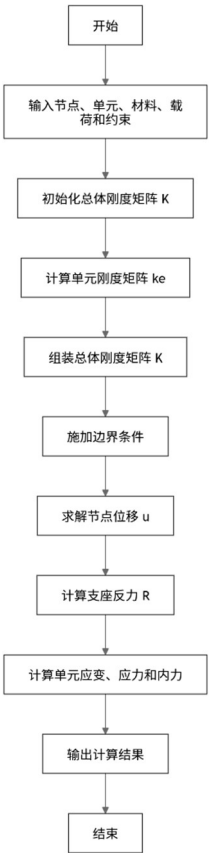


图 3-2 算法流程图

6、程序流程图

本流程图展示了一维杆有限元程序的整体计算过程。程序首先输入节点坐标、单元连接关系、材料参数、载荷和边界条件；然后初始化总体刚度矩阵，并通过单元循环依次计算每个单元的长度和单元刚度矩阵。随后按照单元节点编号，将各单元刚度矩阵组装到总体刚度矩阵中。完成组装后，程序施加位移边界条件，将自由自由度与约束自由度分开，并求解未知节点位移。MATLAB 中的反斜杠 $A \setminus B$ 可用于求解 $A \cdot x = B$ 形式的线性方程组。最后，程序根据节点位移计算支座反力、单元应变、应力和内力，并输出计算结果，见图 3-3。

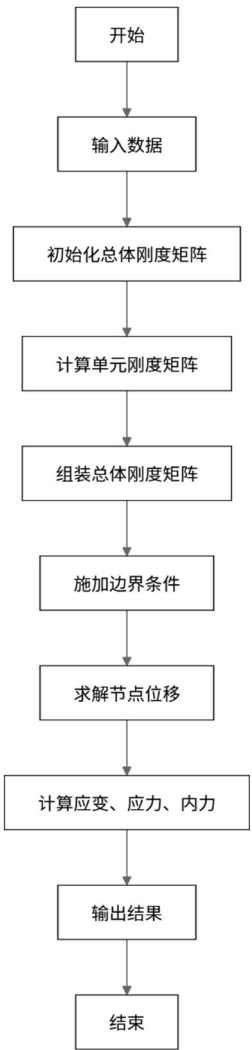


图 3-3 项目流程图

7、程序源代码

程序源代码如下。为便于阅读，代码按照 MATLAB 语法进行了颜色区分，注释、字符串、关键字和数字均采用不同颜色显示。

```
%% 一维拉伸杆有限元程序
clear; clc;

fprintf('一维拉伸杆有限元程序\n');
```

```

fprintf('请统一单位，例如 N-mm 或 N-m\n\n');

x = input('请输入节点坐标列向量，例如 [0;200;400;600] = ');

elem = input('请输入单元连接矩阵，例如 [1 2;2 3;3 4] = ');

nnode = length(x);
nelem = size(elem, 1);

E = input('请输入弹性模量 E，例如 200000 或 200000*ones(nelem,1) = ');

A = input('请输入截面面积 A，例如 200 或 200*ones(nelem,1) = ');

F = input('请输入节点载荷列向量，例如 [0;0;0;20000] = ');

fixedDof = input('请输入约束节点编号，例如 1 或 [1 3] = ');

fixedVal = input('请输入对应约束位移，例如 0 或 [0;0] = ');

if isscalar(E)
    E = E * ones(nelem, 1);
end

if isscalar(A)
    A = A * ones(nelem, 1);
end

if length(E) ~= nelem
    error('E 的个数必须等于单元数');
end

if length(A) ~= nelem
    error('A 的个数必须等于单元数');
end

if length(F) ~= nnode
    error('载荷向量 F 的长度必须等于节点数');
end

if length(fixedDof) ~= length(fixedVal)
    error('约束节点编号 fixedDof 和约束位移 fixedVal 的数量必须一致');
end

K = zeros(nnode, nnode);

for e = 1:nelem

    node1 = elem(e,1);
    node2 = elem(e,2);

    x1 = x(node1);
    x2 = x(node2);

    Le = x2 - x1;

    if Le <= 0
        error('单元 %d 的长度小于或等于 0，请检查节点坐标或单元连接关系', e);
    end
end

```

```

end

ke = E(e) * A(e) / Le * [1 -1; -1 1];

dof = [node1 node2];

for i = 1:2
    for j = 1:2
        K(dof(i), dof(j)) = K(dof(i), dof(j)) + ke(i,j);
    end
end
end

allDof = 1:nnode;
freeDof = setdiff(allDof, fixedDof);

Kff = K(freeDof, freeDof);
Kfc = K(freeDof, fixedDof);
Ff = F(freeDof);

uc = fixedVal(:);

uf = Kff \ (Ff - Kfc * uc);

u = zeros(nnode, 1);
u(freeDof) = uf;
u(fixedDof) = uc;

R = K * u - F;

strain = zeros(nelem,1);
stress = zeros(nelem,1);
force = zeros(nelem,1);
Le_all = zeros(nelem,1);

for e = 1:nelem

    node1 = elem(e,1);
    node2 = elem(e,2);

    x1 = x(node1);
    x2 = x(node2);

    Le = x2 - x1;
    Le_all(e) = Le;

    ue = [u(node1); u(node2)];

    strain(e) = [-1/Le 1/Le] * ue;

    stress(e) = E(e) * strain(e);

    force(e) = stress(e) * A(e);
end

nodeID = (1:nnode)';
nodeResult = table(nodeID, x, u, ...
    'VariableNames', {'节点编号', '坐标', '位移'});

```

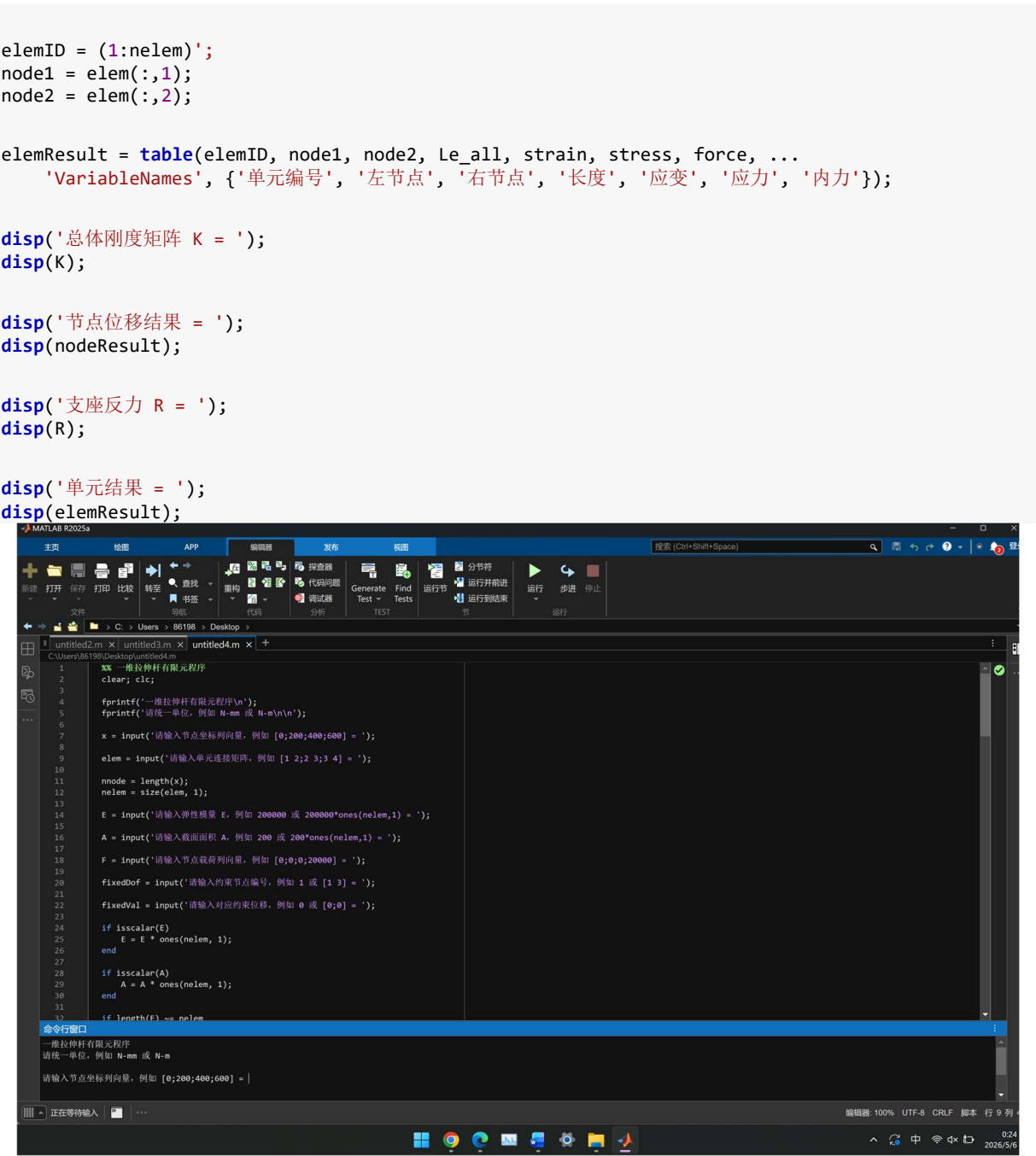


图 3-4 源代码截图

8、程序主要变量说明

表中列出了程序中使用的主要变量及其作用，保留有限元建模、刚度矩阵组装、边界条件处理、方程求解和结果输出中的核心变量。

表 3-1 一维拉伸杆有限元程序主要变量说明

变量名	含义	简要说明
x	节点坐标向量	存放各节点的位置坐标，用于计算单元长度。
elem	单元连接矩阵	表示每个单元由哪两个节点组成。

变量名	含义	简要说明
nnode	节点总数	表示模型中的节点数量。
nelem	单元总数	表示模型中的单元数量。
E	弹性模量	材料参数，用于计算刚度和应力。
A	截面面积	杆件横截面积，用于计算刚度和内力。
F	节点载荷向量	存放各节点所受外力。
fixedDof	约束节点编号	表示被约束位移的节点。
fixedVal	约束位移值	表示约束节点的已知位移，固定端通常为 0。
K	总体刚度矩阵	由各单元刚度矩阵组装得到。
Le	单元长度	由单元两端节点坐标差计算得到。
ke	单元刚度矩阵	表示单个单元的刚度关系。
dof	单元自由度编号	用于将单元刚度矩阵组装到总体刚度矩阵中。
freeDof	自由自由度编号	表示未被约束、需要求解的节点自由度。
Kff	自由自由度刚度矩阵	用于求解未知节点位移。
Ff	自由自由度载荷向量	与自由自由度对应的外载荷。
u	总节点位移向量	存放所有节点的位移结果。
R	支座反力向量	由总体刚度矩阵和位移计算得到。
strain	单元应变	由单元两端节点位移差除以单元长度得到。
stress	单元应力	由胡克定律计算得到。
force	单元内力	由单元应力乘以截面面积得到。
Le_all	单元长度汇总向量	存放所有单元长度。
nodeResult	节点结果表	输出节点编号、坐标和位移。
elemResult	单元结果表	输出单元编号、长度、应变、应力和内力。

9、算例结果

本次程序验证采用任务 2 中的一维等截面直杆受拉算例。杆件长度为 600 mm，截面面积为 200 mm²，弹性模量为 200 GPa = 200000 N/mm²，右端受到轴向拉力 20 kN = 20000 N，左端固定。任务 2 手算中将杆件划分为 2 个等长单元，计算得到右端位移为 0.30 mm，各单元应变为 0.0005，各单元应力为 100 MPa。

本程序验证时将同一根杆划分为 3 个等长单元。由于杆件为等截面直杆，材料均匀，且只受右端轴向集中拉力作用，所以杆内轴力沿全长保持不变，位移沿杆长呈线性分布。因此，虽然程序中的单元数量与任务 2 手算中的单元数量不同，但右端总伸长量和单元应力应保持一致。

程序计算得到的节点位移为：u1 = 0 mm，u2 = 0.10 mm，u3 = 0.20 mm，u4 = 0.30 mm。

程序计算得到的单元应变、应力和内力为： $\epsilon_1 = \epsilon_2 = \epsilon_3 = 0.0005$ ， $\sigma_1 = \sigma_2 = \sigma_3 = 100$ MPa， $N_1 = N_2 = N_3 = 20000$ N。

与任务 2 手算结果相比，程序计算得到的右端位移同样为 0.30 mm，单元应力同样为 100 MPa。根据材料力学解析公式，等截面轴向拉杆的总伸长量和应力分别为：

$$\Delta L = \frac{FL}{EA} \quad (3-2)$$

$$\sigma = \frac{F}{A} \quad (3-3)$$

代入本题数据可得 $\Delta L = 0.30$ mm， $\sigma = 100$ MPa，这与程序结果和手算结果均一致。相关轴向拉伸变形关系中，杆件伸长量与载荷和长度成正比，与弹性模量和截面面积成反比。

因此可以说明，本程序能够正确完成一维杆有限元计算。由于本题的真实位移场本身为线性分布，而一维二节点线性杆单元能够准确表示这种线性位移场，所以无论划分为 2 个单元还是 3 个单元，最终右端位移和单元应力结果都与手算及解析解一致。

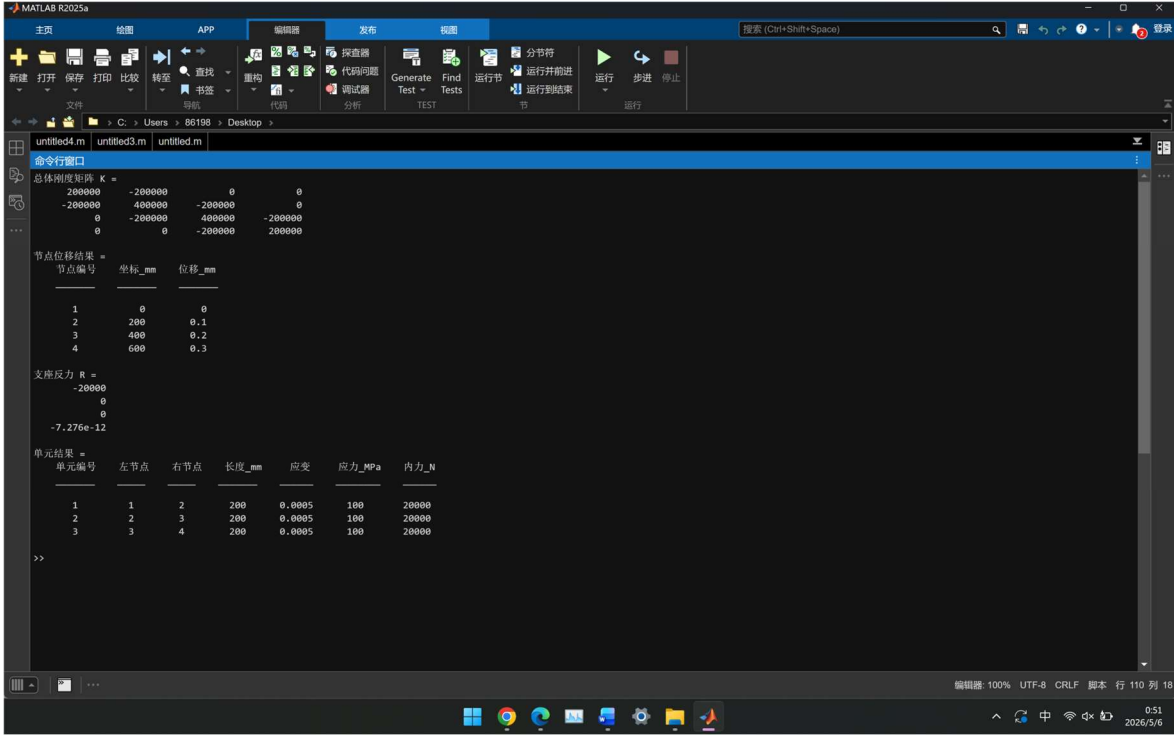


图 3-5 算例输出结果

9.1 测试算例

算例 1：三单元等截面直杆受拉问题。一根一维等截面直杆，左端固定，右端受到轴向拉力。杆长为 $L = 600$ mm，截面面积为 $A = 200$ mm²，弹性模量为 $E = 200$ GPa，右端拉力为 $F = 20$ kN。采用 N-mm 单位制，将杆件划分为 3 个等长一维线性杆单元，利用程序求解节点位移、单元应变、应力和内力。

```

x = [0; 200; 400; 600];
elem = [1 2; 2 3; 3 4];
E = 200000;
  
```



```
A = 200;  
F = [0; 0; 0; 20000];  
fixedDof = 1;  
fixedVal = 0;
```

理论结果为： $u_1 = 0 \text{ mm}$ ， $u_2 = 0.10 \text{ mm}$ ， $u_3 = 0.20 \text{ mm}$ ， $u_4 = 0.30 \text{ mm}$ ； $\sigma_1 = \sigma_2 = \sigma_3 = 100 \text{ MPa}$ 。

算例 2：二单元等截面直杆受拉问题。一根一维等截面直杆，左端固定，右端受到轴向拉力。杆长为 $L = 600 \text{ mm}$ ，截面面积为 $A = 200 \text{ mm}^2$ ，弹性模量为 $E = 200 \text{ GPa}$ ，右端拉力为 $F = 20 \text{ kN}$ 。采用 N-mm 单位制，将杆件划分为 2 个等长一维线性杆单元，利用程序求解节点位移、单元应变、应力和内力，并与手算结果进行比较。

```
x = [0; 300; 600];  
elem = [1 2; 2 3];  
E = 200000;  
A = 200;  
F = [0; 0; 20000];  
fixedDof = 1;  
fixedVal = 0;
```

理论结果为： $u_1 = 0 \text{ mm}$ ， $u_2 = 0.15 \text{ mm}$ ， $u_3 = 0.30 \text{ mm}$ ； $\sigma_1 = \sigma_2 = 100 \text{ MPa}$ 。

由表 3-2 可知，程序计算得到的右端位移、单元应变、单元应力、单元内力和支座反力均与任务 2 手算结果或材料力学解析解一致。虽然两种计算中的单元划分数量不同，但杆件为等截面轴向受拉杆，真实位移场为线性分布，因此有限元程序能够得到一致的总伸长量和应力结果。

表 3-2 程序结果与手算/解析结果对比表

对比项目	任务 2 手算/解析结果	程序计算结果	误差	是否一致	说明
右端位移/总伸长量	0.30 mm	0.30 mm	0	是	均与 $\Delta L = FL/EA$ 的解析解一致
节点位移分布	$u_1=0 \text{ mm}$, $u_2=0.15 \text{ mm}$, $u_3=0.30 \text{ mm}$	$u_1=0 \text{ mm}$, $u_2=0.10 \text{ mm}$, $u_3=0.20 \text{ mm}$, $u_4=0.30 \text{ mm}$	—	合理	单元数不同，中间节点位置不同，但位移均呈线性分布
单元应变	$\varepsilon_1=\varepsilon_2=0.0005$	$\varepsilon_1=\varepsilon_2=\varepsilon_3=0.0005$	0	是	各单元应变相同
单元应力	$\sigma_1=\sigma_2=100 \text{ MPa}$	$\sigma_1=\sigma_2=\sigma_3=100 \text{ MPa}$	0	是	均与 $\sigma = F/A$ 的解析解一致
单元内力	$N_1=N_2=20000 \text{ N}$	$N_1=N_2=N_3=20000 \text{ N}$	0	是	杆内轴力沿全长保持不变
支座反力	$R_1=-20000 \text{ N}$	$R_1=-20000 \text{ N}$	0	是	支座反力与外力大小相等、方向相反
总体结论	有限元解与解析解一致	程序结果与手算/解析结果一致	—	是	程序能够正确完成一维

对比项目	任务 2 手算/ 解析结果	程序计算结果	误差	是否一致	说明
					杆有限元计算

10、关键语句解释

```
x = input('请输入节点坐标列向量，例如 [0;200;400;600] = ');
```

该语句用于输入节点坐标。节点坐标决定每个节点在杆件轴线方向上的位置，是计算单元长度的基础。

```
elem = input('请输入单元连接矩阵，例如 [1 2;2 3;3 4] = ');
```

该语句用于输入单元连接关系。矩阵中每一行表示一个单元，例如 [1 2] 表示该单元由节点 1 和节点 2 组成。

```
nnode = length(x);
```

该语句用于计算节点总数。由于一维杆单元每个节点只有一个轴向位移自由度，所以节点数也对应总体自由度数。

```
nelem = size(elem, 1);
```

该语句用于计算单元总数。elem 矩阵有多少行，就表示有多少个单元。

```
if isscalar(E) 和 if isscalar(A)
```

这两段语句用于判断输入的弹性模量和截面面积是不是单个数。如果输入的是单个数，程序会自动将其扩展成每个单元对应的参数。这样既可以处理等截面、同材料杆件，也可以扩展到不同单元参数的情况。

```
K = zeros(nnode, nnode);
```

该语句用于初始化总体刚度矩阵。因为一维杆每个节点只有一个位移自由度，所以总体刚度矩阵的大小为 $nnode \times nnode$ 。MATLAB 中 zeros 可用于创建全零数组。

```
for e = 1:nelem
```

该语句表示对所有单元进行循环计算。每循环一次，程序就处理一个单元，包括读取节点、计算长度、形成单元刚度矩阵并进行组装。

```
node1 = elem(e,1); node2 = elem(e,2);
```

这两句用于读取第 e 个单元的左右节点编号。根据这两个节点编号，程序可以找到对应节点坐标和位移。

```
Le = x2 - x1;
```

该语句用于计算单元长度。对于一维杆单元，单元长度等于右端节点坐标减去左端节点坐标。

```
ke = E(e) * A(e) / Le * [1 -1; -1 1];
```

该语句用于形成一维杆单元刚度矩阵。其中 $E(e)$ 是第 e 个单元的弹性模量， $A(e)$ 是第 e 个单元的截面面积， Le 是单元长度。该矩阵反映单元两端节点力与节点位移之间的关系。

```
dof = [node1 node2];
```

该语句用于确定当前单元对应的总体自由度编号。由于一维杆每个节点只有一个位移自由度，所以单元的自由度编号就是它的两个节点编号。

```
K(dof(i), dof(j)) = K(dof(i), dof(j)) + ke(i,j);
```

该语句用于将单元刚度矩阵组装到总体刚度矩阵中。不同单元如果连接到同一个节点，其刚度贡献需要在总体刚度矩阵的对应位置进行叠加。

```
freeDof = setdiff(allDof, fixedDof);
```

该语句用于找出自由自由度。allDof 表示全部自由度，fixedDof 表示被约束的自由度，两者相减后得到需要求解的未知位移自由度。

```
Kff = K(freeDof, freeDof);
```

该语句从总体刚度矩阵中提取自由自由度对应的子矩阵，用于建立未知节点位移的求解方程。

```
Kfc = K(freeDof, fixedDof);
```

该语句提取自由自由度与约束自由度之间的刚度耦合项。它用于考虑已知约束位移对自由节点方程的影响。

```
uc = fixedVal(:);
```

该语句将约束位移转换为列向量，保证后续矩阵运算维度一致。

```
uf = Kff \ (Ff - Kfc * uc);
```

该语句是求解未知节点位移的核心语句。其中 Kff 是自由自由度刚度矩阵，Ff - Kfc * uc 是修正后的自由自由度载荷向量。MATLAB 中 A\B 用于求解 $A*x = B$ 形式的线性方程组，因此这里求得的是未知位移 uf。

```
u(freeDof) = uf; u(fixedDof) = uc;
```

这两句用于组合完整的节点位移向量。自由节点位移由程序求解得到，约束节点位移由输入条件给定。

```
R = K * u - F;
```

该语句用于计算支座反力。总体内力 $K*u$ 减去外载荷 F 后，得到结构各节点的反力，其中约束节点处的反力就是支座反力。

```
strain(e) = [-1/Le 1/Le] * ue;
```

该语句用于计算第 e 个单元的应变。对于一维二节点杆单元，应变等于右节点位移减去左节点位移，再除以单元长度。

```
stress(e) = E(e) * strain(e);
```

该语句用于计算单元应力。它根据胡克定律，应力等于弹性模量乘以应变。

```
force(e) = stress(e) * A(e);
```

该语句用于计算单元轴向内力。单元内力等于单元应力乘以截面面积。

```
nodeResult = table(nodeID, x, u, ...);
```

该语句用于生成节点结果表，包括节点编号、节点坐标和节点位移。MATLAB 中 table 函数可由输入数组创建表格。

```
elemResult = table(elemID, node1, node2, Le_all, strain, stress, force, ...);
```

该语句用于生成单元结果表，包括单元编号、左右节点、单元长度、应变、应力和内力，使程序输出结果更加清楚。

11、AI 辅助代码记录

本任务中使用 AI 辅助完成了以下内容：

1. 根据任务要求规划框架；
2. 检查并补充单元刚度矩阵、总体刚度组装、边界条件处理等关键代码；
3. 解释 Kff\(...)、setdiff、table 等 MATLAB 语句的作用；
4. 帮助整理测试算例和结果验证表。

本人对 AI 生成内容进行了以下修改：

1. 根据任务要求改为手动输入数据版本；
2. 将算例单位统一为 N-mm 制；
3. 将程序输出结果与手算结果进行对比。

12、总结

本任务将一维杆有限元手算过程转化为 MATLAB 程序，实现了从输入数据、计算单元刚度、组装总体刚度矩阵、施加边界条件、求解节点位移到计算单元应力和内力的完整过程。通过两个算例验证，程序计算结果与解析解或手算结果一致，说明该程序能够正确完成一维杆有限元的基本求解。